

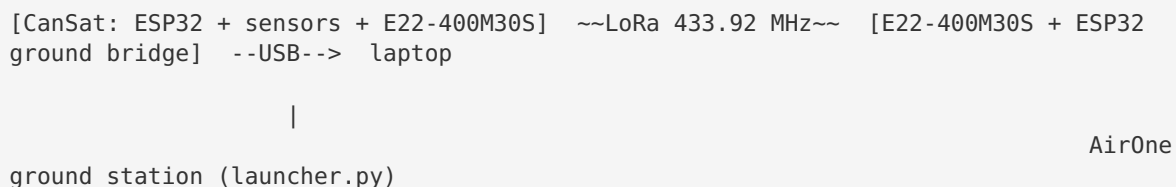
AirOne — Connecting a Real CanSat (E22-400M30S SPI + USB ground bridge)

This guide explains how to receive **live telemetry** from a physical CanSat on your laptop. The CanSat and the ground station each use an **EBYTE E22-400M30S** (Semtech SX1268, **SPI-only**) driven by an ESP32. The ground ESP32 runs `airone_ground_bridge/` and appears on your laptop as a USB serial port. The guide covers wiring, radio parameters, launching the ground station in live mode, and troubleshooting.

Status. The SPI radio driver and the ground bridge compile and pass the host tests. They have **not** been tested on hardware yet. The radio pin assignment below is an **assumption** — check it against your PCB before you power anything up.

Scientific-honesty guarantee. The ground station never fabricates telemetry. If no radio is found it reports an explicit `NOT_CONFIGURED` state and streams nothing. Simulation (`--simulate`) is an isolated testing/training mode only; real telemetry is always authoritative and simulated packets are tagged `SIMULATED` .

1. Signal chain overview



- The **flight radio** is driven over SPI by RadioLib in `airone_cansat/airone_radio.h` . Every 2 Hz sample is logged in full (JSON frame) to MicroSD/W25Q128. Over the air the CanSat sends a **compact frame** (payload type `0x10` , ≤ 255 B, HMAC-signed when a link key is set). The 10 % duty-cycle limiter sets the real air rate: about one frame every ~ 1.6 s at SF7/BW250. That figure is an estimate.
- The **ground bridge** (`airone_ground_bridge/`) receives the compact frame and verifies CRC/HMAC. It drops duplicate sequence numbers, expands the frame to the normal JSON sensor payload (adding ground-measured `radio_rssi` / `radio_snr`) and writes a `PT_SENSOR_DATA` frame to USB serial at **115200 baud**. The ground station sees exactly the same binary framing as before.
- Both radios must share the **same LoRa parameters** (section 4). The E22 has no UART mode, no M0/M1 pins and no module-level address/channel configuration.

2. Ground-side wiring (E22-400M30S ↔ ground-bridge ESP32)

Any ESP32 dev board with a CP210x/CH340 USB bridge works. The ground station auto-discovers those USB chips. The bridge uses the same pin defaults as the flight board, and you can override each one with a `-D` build flag (see `airone_radio.h`).

E22-400M30S pin	ESP32 pin	Notes
VCC	supply per the E22-400M30S datasheet (≈ 30 dBm PA draws several hundred mA in TX)	Check the datasheet before connecting
GND	GND	
SCK	GPIO18	VSPI
MISO	GPIO19	VSPI
MOSI	GPIO23	VSPI
NSS	GPIO13	<code>RADIO_PIN_NSS</code>
BUSY	GPIO27	<code>RADIO_PIN_BUSY</code>
DIO1	GPIO35	<code>RADIO_PIN_DIO1</code> (input-only pin, IRQ)
NRST	not connected	<code>RADIO_PIN_NRST=-1</code> ; connect one and set the flag if you want hardware reset
RXEN	GPIO14	<code>RADIO_PIN_RXEN</code> - RF switch, RX path
TXEN	GPIO33	<code>RADIO_PIN_TXEN</code> - RF switch, TX path
ANT	433 MHz antenna	Never key the PA without an antenna/load attached

The E22-400M30S uses 3.3 V logic. Do not drive its pins from 5 V logic (ESP32 GPIOs are 3.3 V, so a direct connection is fine).

Flash the bridge with `arduino-cli compile/upload -b esp32:esp32:esp32 airone_ground_bridge`. If the flight unit uses an HMAC link key, build with the same key (`AIRONE_LINK_KEY_HEX`). Build with `-DBRIDGE_DEBUG=1` to get

per-packet text in a serial monitor. Leave it off when the ground station is reading the port, because text breaks the binary stream.

3. Flight-side wiring (ESP32-WROVER ↔ E22-400M30S)

Defaults used by `airone_cansat/airone_cansat.ino` / `airone_radio.h`. The radio shares the VSPI bus with the BMI270 (CS GPIO15), MicroSD (CS GPIO5) and W25Q128 (CS GPIO4). A FreeRTOS mutex (`airone_spibus.h`) serialises all bus access.

ESP32 pin	E22 pin	Notes
GPIO18 / 19 / 23	SCK / MISO / MOSI	shared VSPI
GPIO13	NSS	driven HIGH at boot, before any SPI traffic
GPIO27	BUSY	
GPIO35	DIO1	input-only, TX-done/RX-done IRQ
—	NRST	not connected (assumed)
GPIO14	RXEN	driven LOW at boot
GPIO33	TXEN	driven LOW at boot
3.3 V	VCC	
GND	GND	

Pin assumption — verify. An older V7.1 map put RXEN on GPIO32, NRST on GPIO16 and DIO1 on GPIO17. On V8.0 those pins clash: GPIO32 is the TPS3823 WDI, and GPIO16/17 are used by the WROVER's PSRAM. The firmware therefore re-uses the old UART-E22 pads: M0→RXEN (14), M1 pad→NSS (13), AUX→DIO1 (35). If your PCB is routed differently, set `-DRADIO_PIN_*`.

The MAX-M10S GNSS is on UART2 (ESP32 RX = GPIO25, TX = GPIO26). I²C sensors are on SDA=21 / SCL=22 behind a PCA9548A mux (0x70). The full pin map is in the firmware `README.md`.

4. LoRa parameters must match (bridge USB baud is fixed at 115200)

Both sketches take their RF settings from `airone_radio.h`. The run-script checks that the copies are identical. If you change a parameter, change it in both builds:

Parameter	Default	Flag
Frequency	433.92 MHz	<code>RADIO_FREQ_MHZ</code>
Bandwidth	250 kHz	<code>RADIO_BW_KHZ</code>
Spreading factor	7	<code>RADIO_SF</code>
Coding rate	4/5	<code>RADIO_CR</code>
Sync word	0x12 (private)	<code>RADIO_SYNC_WORD</code>
Preamble	8 symbols	<code>RADIO_PREAMBLE</code>
TCXO	1.8 V	<code>RADIO_TCXO_V</code>
SX1268 output	−9 dBm (+ ~21 dB E22 PA ≈ +12 dBm ≈ 10 mW e.r.p. with a 0 dBi antenna, estimate)	<code>RADIO_POWER_DBM</code>
Duty cycle	10 %	<code>RADIO_DUTY_PCT</code>

UK IR2030 (433.05–434.79 MHz) allows **10 mW e.r.p.** and a **10 %** duty cycle. The default power is a calculated estimate, not a measurement. Measure it, or check it against the E22 datasheet's PA gain and your antenna gain, before you fly.

The bridge always talks to the laptop at **115200 baud**. Because the bridge's USB port runs at a fixed baud, `--force-baud` is recommended:

```
python launcher.py --serial-port /dev/ttyUSB0 --baud 115200 --force-baud
```

5. Finding the serial port

- **Linux:** usually `/dev/ttyUSB0` (FTDI/CP210x) or `/dev/ttyACM0`.
List devices: `ls /dev/ttyUSB* /dev/ttyACM*` or `dmesg | grep tty` after plugging in.
You may need permission: `sudo usermod -aG dialout $USER` then re-login.
- **macOS:** `/dev/tty.usbserial-XXXX` or `/dev/tty.SLAB_USBtoUART`.
- **Windows:** `COM3`, `COM5`, ... (see Device Manager → Ports).

Auto-discovery: if you omit `--serial-port`, AirOne scans for known USB-serial bridges (CP210x `0x10C4`, CH340 `0x1A86`, FTDI `0x0403`) and picks the first match.

6. Launching the ground station in live mode

Set the mandatory JWT secret (≥ 32 chars) and run the launcher:

```
export AIRONE_JWT_SECRET="your-long-random-secret-at-least-32-characters"

# Auto-discover the radio:
python launcher.py

# Or specify the port/baud explicitly (recommended for the ground bridge):
python launcher.py --serial-port /dev/ttyUSB0 --baud 115200 --force-baud
```

On startup the launcher prints the explicit link state, one of:

- `LIVE (/dev/ttyUSB0 @ 115200 baud)` — receiving live telemetry.
- `NOT_CONFIGURED (no CanSat radio found ...)` — nothing connected; **no data is fabricated**. Plug in the radio (or use `--simulate` for isolated testing).
- `DISABLED (telemetry.enabled=false)` — link intentionally turned off.

The live link status is also available in the health report / API and updates `bytes_received`, `packets_received`, `crc_errors`, and `reconnect_count`.

Configuration reference (`config/default_config.yaml`)

```
telemetry:
  enabled: true           # false = do not start the live link
  serial_port: null       # null = auto-discover; or "/dev/ttyUSB0" / "COM5"
  baud_rate: 115200      # ground bridge USB baud
  force_baud: false       # true = open directly at baud_rate (recommended for the
                           bridge)
```

CLI flags override config. `AIRONE_SERIAL_PORT` env var overrides the config `serial_port` (but is overridden by `--serial-port`).

7. What good telemetry looks like

Each frame the CanSat sends is CRC-protected. When frames arrive:

- `packets_received` climbs and `crc_errors` stays low.
 - Valid frames flow through the 20-stage pipeline and appear in the GUI/API.
 - Corrupted frames are counted as CRC errors and **discarded** (never accepted with fabricated values).
-

8. Troubleshooting

Symptom	Likely cause / fix
<code>NOT_CONFIGURED</code> but the bridge is plugged in	Wrong/aliased port. Pass <code>--serial-port</code> explicitly and check <code>ls /dev/ttyUSB*</code> . On Linux, add yourself to <code>dialout</code> .
Flight serial log shows <code>[RADIO] SX1268 init FAILED (-2) chip not found</code>	<code>RADIOLIB_ERR_CHIP_NOT_FOUND</code> : wrong NSS/BUSY pin, no power, or SPI wiring fault. Check the pin flags and the 3.3 V rail.
<code>init FAILED (-705) BUSY never released / (-703) invalid TCXO voltage</code>	TCXO voltage or BUSY line issue. Confirm <code>RADIO_TCXO_V</code> against the E22 datasheet and that BUSY goes to the configured GPIO.
Type <code>L</code> in the flight serial console	Prints storage status and radio status (<code>tx</code> , <code>skipped(duty/busy)</code> , <code>errors</code> , <code>last_err</code>). Use it to see whether frames are actually being sent.
Bridge receives nothing	LoRa parameter mismatch (frequency/SF/BW/sync word/CR). Both builds must use the same <code>airone_radio.h</code> . Also check the RXEN/TXEN wiring, because an RF switch left in the wrong state means no RX.
Bridge receives frames but forwards none	HMAC key mismatch (frames fail verification) or duplicate sequence numbers. Rebuild with <code>-DBRIDGE_DEBUG=1</code> and watch the <code>#RX bad frame</code> lines.
Port opens but <code>packets_received</code> stays 0	Use <code>--baud 115200 --force-baud</code> . Confirm the CanSat is powered and transmitting (<code>L</code> shows <code>tx=</code> counting up).
Updates arrive about every 1.6 s, not 2 Hz	Expected. The 10 % duty-cycle limiter sets the air rate. The full 2 Hz record is on MicroSD/flash.
Many <code>crc_errors</code>	RF interference / weak link. Check antennas and distance. Note that the bridge already discards bad frames before USB.
Data arrives then stops	Link dropped. The receiver auto-reconnects with backoff. Check power/USB. <code>reconnect_count</code> will increase.
<code>pyserial</code> not installed	<code>pip install -r requirements.txt</code> (installs <code>pyserial</code>).

Symptom	Likely cause / fix
Bytes look garbled in a serial monitor	That's the binary frame format, which is expected. Use the AirOne ground station, or build the bridge with <code>BRIDGE_DEBUG=1</code> .
Permission denied on <code>/dev/ttyUSB0</code> (Linux)	<code>sudo usermod -aG dialout \$USER</code> , then log out/in.

9. Testing without hardware (isolated simulation)

When you have no radio at hand, exercise the entire real pipeline with the physics-based simulator. Simulated packets use the identical binary framing and are tagged `SIMULATED` :

```
python launcher.py --simulate --sim-loop
```

If a live link is also up, real telemetry remains authoritative; the launcher warns that simulation is running alongside it.